

Prerequisite Gate — Async Python (before Document 09)

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Not a numbered document — a short, focused checkpoint. LangGraph's way of running things (streaming, running steps at the same time, `ainvoke / astream`) assumes you're comfortable with `async / await`. This is the first place in the curriculum where you actually need it, so it's taught here instead of earlier, where it would have no real use case yet.

Prerequisites

[08_rag](#)

How to Read & Practice This Gate

- **What:** running several things at once in Python — `async / await`, the event loop, `asyncio.gather`.
- **Why:** LangGraph, and any real production API, depend on this. Skip it, and Doc09 will feel like magic instead of something you understand.
- **When:** any time you're making several separate calls (API calls, database queries) that don't depend on each other's answers.
- **How to practice:** 1. Read the Real Python walkthrough in full — this idea really does need the longer explanation, not just the short official-docs version. 2. Do the **Basic** exercise closed-book. 3. Do the **Intermediate/Real-world** timing comparison — the actual numbers, not just the theory, are what make this click. 4. Do the **Failure** exercise on purpose — breaking things at running-together-ness on purpose teaches this faster than just reading about it. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, explain out loud why `async` helps for waiting-on-something work, and does nothing for heavy-computing work. If you can't, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#)

The Story — what this document is actually building

Picture this: Doc09's LangGraph agent needs to call three tools to answer one question — maybe a search, a lookup, and a calculation. Nothing about those three calls depends on the other two's answers. Run them one after another with plain, blocking Python, and you wait for all three, back to back — a 2-second call, another 2-second call, another 2-second call, six seconds total, even though the computer was just sitting idle waiting for a network reply the whole time.

That idle waiting time is the whole story of this gate. `async / await` doesn't make Python faster at computing — it lets Python use the time it would otherwise spend just waiting on a network reply to go start another task instead. One thread, one event loop, switching between tasks only at the moments one of them is waiting on something. `asyncio.gather` is the tool that actually captures that benefit: instead of `await a(); await b(); await c()` (one after another, six seconds), you write `await asyncio.gather(a(), b(), c())` (all three started at once, roughly two seconds — however long the slowest one takes alone).

This has no Build Task of its own — it's a checkpoint, not a project. The reason it exists as its own short document, right here and not earlier, is that Doc09's LangGraph relies on this directly: streaming, running nodes in parallel, and the async client patterns you'll use from here through the rest of the curriculum all assume you're already comfortable with `async / await`. Get this solid now, and Doc09 will read like ordinary code. Skip it, and Doc09 will feel like magic you can't debug.

Core Concepts (read this first — everything you need is here)

The event loop: one thread, many waiting tasks

Python's `asyncio` runs a single-threaded **event loop** that juggles several tasks by switching between them whenever one is *waiting* on something (a network reply, a timer) instead of actively doing work. A coroutine (an `async def` function) is a function that can pause at an `await` point, hand control back to the loop, which goes and makes progress on something else, then comes back to this function once whatever it was waiting for is ready. **Why this is different from using threads:** no separate operating-system threads, and none of the locking complexity that comes with them — running things "at the same time" comes from cooperatively switching at `await` points, all on one thread. **How `await` really behaves:** calling an async function *without* `await` doesn't actually run it — it just creates a coroutine object sitting quietly in memory. `await` is what actually schedules and runs it, which is exactly why "I called the function, but nothing happened" is such a common mistake.

Why async helps waiting-on-something work, and does nothing for heavy-computing work

While your code is waiting for a network reply, the CPU isn't doing anything — it's just sitting idle, waiting. Async lets the event loop use that idle time to make progress on a *different* task instead of just sitting there. **Why this doesn't help heavy-computing work** (lots of math, not waiting on anything outside): there's no idle time to use — the CPU is already fully busy. Switching to another

task wouldn't speed anything up — it would just interleave two computations on the same single thread, taking the same total time (or a bit longer, from the switching itself). **When to actually use `async`:** any code making several separate network calls (several LLM calls, several tool calls, several database queries) that don't depend on each other's answers.

`asyncio.gather` : running things at the same time instead of one after another

`await a(); await b(); await c()` runs three tasks one after another — the total time is roughly the sum of all three. `await asyncio.gather(a(), b(), c())` starts all three *at the same time* and waits for all of them to finish — the total time is roughly however long the *slowest* one takes alone, not the sum of all three. **Why this matters, in real terms, for LLM apps:** if you need to call three separate tools, or check three separate sources of data, running them at the same time instead of one after another can turn a 6-second wait into a 2-second wait — for basically no extra cost, just writing `gather` instead of three separate `await` lines.

Sync vs. async client: same tool, different way of running

The OpenAI library ships two client types — `OpenAI` (runs one thing at a time, waits) and `AsyncOpenAI` (runs things at the same time) — with almost the same method names. The async version just needs `await` in front of each call, and has to run inside an `async def` function under an event loop. **Why both exist:** a simple terminal script (Project 1) doesn't need the extra complexity of async — doing one thing at a time is fine. A production API service handling many requests at once (Doc12) benefits a lot from async, since it can work on other requests while waiting on a slow LLM call, instead of freezing everything.

Common mistakes, and why they're sneaky

Forgetting `await` — as above, the coroutine gets created but never actually runs. This often fails *silently* (no error, just nothing happens), instead of crashing, which makes it unusually hard to notice.

Blocking the event loop — calling a normal, waiting-style function (like the sync `OpenAI` client, or `time.sleep()`) from inside an `async def` function freezes the **whole** event loop for that time — killing the "at the same time" behavior you thought you had, silently — no error, everything just runs one-at-a-time again, for reasons that aren't obvious from just reading the code.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [Python docs — asyncio](#) — the official reference; skim for the shape of it, don't try to memorize every function.
- [Real Python — Async IO in Python: A Complete Walkthrough](#) — a longer, clearer explanation.

Practice Exercises

Setup for this document's practice code: work inside `08b_async_prereq/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install openai`.

How to run each exercise: save it as its own small script — `practice_basic.py`, `practice_intermediate.py`, and so on, matching the levels below — and run it directly: `python practice_basic.py`. Keep each one runnable on its own; don't chain them into one file.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#)

Basic — your first coroutine

- **What:** an `async def` function that awaits `asyncio.sleep(1)` and returns a value, called with `asyncio.run()`.
- **Why:** you need to see, once, that `async def` alone doesn't run anything — `await` and `asyncio.run()` are what actually make it go.
- **When you'll hit this for real:** the first time you write any async code at all, right here.
- **How to code it:** `async def wait_and_return(): await asyncio.sleep(1); return "done"`, then `print(asyncio.run(wait_and_return()))`.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — feel the speed difference

- **What:** run 3 API calls one after another with `await`, timed. Then run the same 3 with `asyncio.gather`, timed. Compare.
- **Why:** the numbers, not the theory, are what actually make "running things at the same time" click — this is the exercise that matters most in this whole document.
- **When you'll hit this for real:** any agent making 2+ independent tool calls — this exact pattern is what LangGraph's parallel node execution relies on.
- **How to code it:** write 3 `async def` functions that each `await asyncio.sleep(2)`. Time `await f1(); await f2(); await f3()` with `time.perf_counter()`, then time `await asyncio.gather(f1(), f2(), f3())` — compare the two durations directly.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — convert a real function to async

- **What:** convert one function from Project 1 (Doc04) to use `AsyncOpenAI` instead of the normal client.
- **Why:** this is the exact conversion Doc12's FastAPI service needs for every route that calls the model — better to do it once here, deliberately, than for the first time under pressure later.

- **When you'll hit this for real:** wrapping any agent behind a real API service (Project 5).
- **How to code it:** replace `from openai import OpenAI` with `from openai import AsyncOpenAI`, make the function `async def`, and `await client.chat.completions.create(...)` instead of calling it directly.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — proving `gather` actually waits

- **What:** call `asyncio.gather` with one coroutine that finishes fast and one that sleeps for 5 seconds — confirm `gather` waits for the slow one, and that the fast one's result was ready long before that.
- **Why:** it's easy to assume `gather` returns as soon as the *first* task finishes — it doesn't, and you need to have watched it wait for the slowest one, on purpose.
- **When you'll hit this for real:** any parallel-agent design (Doc11) where one agent is much slower than the others — the whole group waits for it.
- **How to code it:** one coroutine sleeps 0.1s and returns immediately with a printed timestamp, the other sleeps 5s. Run both through `gather`, and print timestamps before/after to see the fast one finished long before `gather` actually returned.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — accidentally blocking the event loop

- **What:** call a normal, waiting-style function from inside an `async def` function on purpose, without `await` or threading, and watch it kill the "at the same time" behavior you expected.
- **Why:** this is the single most common real async bug, and it fails *silently* — no error, just quietly-worse performance — so you need to have seen it once to recognize it later.
- **When you'll hit this for real:** accidentally using the sync `OpenAI` client (instead of `AsyncOpenAI`) inside an async FastAPI route in Project 5.
- **How to code it:** inside an `async def`, call `time.sleep(3)` (not `asyncio.sleep`) instead of awaiting something, run it alongside another coroutine with `gather`, and time it — confirm the total time is now the *sum*, not the max, proving concurrency broke.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Expected Behavior

- You can explain, without notes, why `await asyncio.gather(a(), b(), c())` is faster than `await a(); await b(); await c()` for waiting-on-something work — and why it would *not* be faster for heavy-computing work.
- You can spot, in a piece of code, a call that's silently freezing the event loop.

Break-It / Debug Preview

- A coroutine that's defined but never awaited (it silently does nothing — a very common real mistake).
- A normal, waiting-style call hidden inside an async function that kills "at the same time" behavior without raising any error.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- Event loop basics · `asyncio.gather` vs. one-after-another `await` · sync vs. async client trade-offs · what "blocking the event loop" means and why it's bad.

Move On When

You can correctly explain running-at-the-same-time vs. one-after-another async code, and you've converted at least one real function to async without help. Then move on to [09_langgraph](#).

Stuck? Ask for **Hint 1** through **Hint 2**. Ask for the full solution only if you say "**Show me the solution.**"